

Séance 2

A) PROBABILITÉS

exercice 1:

1) Soient $T_1 \sim \text{Geom}(p)$ et $T_2 \sim \text{Geom}(p')$ (convention: sur $\mathbb{N}^*$)
On suppose $T_1 \neq T_2$.

$$\begin{aligned}
 \text{a) } P(T_2 > k) &= P(T_2 \geq k+1) = \sum_{n=k+1}^{+\infty} P(T_2 = n) \\
 &= \sum_{n=k+1}^{+\infty} p'(1-p')^{n-1} \\
 &= p' \sum_{n=k+1}^{+\infty} (1-p')^{n-1} \\
 &= \frac{p'}{1-p'} \sum_{n=k+1}^{+\infty} (1-p')^n \\
 &= \frac{p'(1-p')^{k+1}}{1-p'} \cdot \sum_{n=0}^{+\infty} (1-p')^n \\
 &= p'(1-p')^k \cdot \frac{1}{1-(1-p')} = (1-p')^k
 \end{aligned}$$

b) Fixons k . Alors $\{T_1 < T_2\} = \{T_1 = k \text{ et } T_2 > k\}$.
Donc en "sommant" sur les k , on a:

$$\{T_1 < T_2\} = \bigcup_{k \geq 1} \{T_1 = k \text{ et } T_2 > k\}.$$

$$\begin{aligned}
 \text{c) } P(T_1 < T_2) &= \sum_{k=1}^{+\infty} P(\{T_1 = k \text{ et } T_2 > k\}) \quad (\text{σ-additivité}) \\
 &= \sum_{k=1}^{+\infty} P(T_1 = k) \cdot P(T_2 > k) \quad (T_1 \neq T_2) \\
 &= \sum_{k=1}^{+\infty} p(1-p)^{k-1} (1-p')^k \\
 &= \frac{p}{1-p} \sum_{k=1}^{+\infty} [(1-p)(1-p')]^k
 \end{aligned}$$

$$= \frac{p(1-p)(1-p')}{(1-p)} \sum_{k=0}^{\infty} (1-p)(1-p')^k$$

$$= p(1-p') \cdot \frac{1}{1-(1-p)(1-p')} \quad (\sum \text{ s\u00e9rie g\u00e9om\u00e9trique})$$

$$= \frac{p(1-p')}{p+p'-pp'} = \frac{p-pp'}{p+p'-pp'} \quad \square$$

2) On note :

- Π : "Mario gagne"
- U : variable de Bernoulli de param\u00e8tre $1/2$
- T_{eq} : nombre de lancers n\u00e9cessaires pour obtenir un 6 avec le d\u00e9 \u00e9quilibr\u00e9.
- T_b : "d\u00e9 truqu\u00e9".

On a : $T_{eq} \sim \text{Geom}(\frac{1}{6})$ et $T_b \sim \text{Geom}(p)$

On admet que $\Pi = \{U=1 \cap T_b < T_{eq}\} \cup \{U=0 \cap T_{eq} < T_b\}$
et $U \perp T_{eq} \perp T_b$. (Tous ind\u00e9pendants).

On a :

$$P(\Pi | U=1) = \frac{P(\Pi \cap \{U=1\})}{P(U=1)} = \frac{P(\{U=1\} \cap \{T_b < T_{eq}\})}{P(U=1)}$$

$$= \frac{P(U=1) \cdot P(T_b < T_{eq})}{P(U=1)} = P(T_b < T_{eq})$$

1a)

$$= \frac{p - \frac{1}{6}p}{p + \frac{1}{6} - \frac{1}{6}p} = \frac{\frac{5}{6}p}{\frac{5}{6}p + \frac{1}{6}} = \frac{5p}{5p+1}$$

b) On cherche désormais $P(M)$.
 $\{U=1\}, \{U=0\}$ forme un système de probabilités complets,
 donc par la formule des probabilités totales,

$$\begin{aligned} P(M) &= P(U=1)P(M|U=1) + P(U=0)P(M|U=0) \\ &= \frac{1}{2} \left(P(M|U=1) + P(M|U=0) \right) \\ &= \frac{1}{2} \left(\frac{S_p}{S_{p+1}} + P(M|U=0) \right) \end{aligned}$$

Or $P(M|U=0) = P(T_{eq} < T_0) = \frac{1/6 - 1/6p}{S_{6p} + 1/6} = \frac{1-p}{S_{p+1}}$

$$\begin{aligned} \text{Donc } P(M) &= \frac{1}{2} \cdot \left(\frac{S_p + 1-p}{S_{p+1}} \right) \\ &= \frac{1}{2} \cdot \frac{4p+1}{S_{p+1}} = \frac{4p+1}{16p+2} \end{aligned}$$

c) On cherche $P(U=1|M)$. En effet, Mario prend le dé
 Draisé si la pièce tombe sur Pile.
 On a :

$$\begin{aligned} P(U=1|M) &= \frac{P(U=1 \cap M)}{P(M)} \quad \text{PROB} \\ &= \frac{P(M|U=1)P(U=1)}{P(M)} = \frac{P(M|U=1)}{2P(M)} \end{aligned}$$

$$= \frac{1}{2} \cdot \frac{S_p}{S_{p+1}} \cdot \left(\frac{2(S_{p+1})}{4p+1} \right) = \frac{S_p}{4p+1} \quad \square$$

exercice 2: Soit $X \sim \text{Po}(\lambda)$, i.e. $P(X=n) = e^{-\lambda} \frac{\lambda^n}{n!}$

a) On pose $Y := X'$.
Par la formule de transfert, on a:

$$\begin{aligned} E[Y] &= \sum_{h \geq 0} h! P(X=h!) && \text{par la formule de transfert} \\ &= \sum_{h \geq 0} h! \cdot e^{-\lambda} \frac{\lambda^h}{h!} = \sum_{h \geq 0} e^{-\lambda} \lambda^h = e^{-\lambda} \sum_{h \geq 0} \lambda^h \\ &= \frac{e^{-\lambda}}{1-\lambda} && \text{pour } |\lambda| < 1 \end{aligned}$$

Question: si $|\lambda| > 1$ peut-on dire que $E[Y] = +\infty$ ou que Y n'admet pas d'espérance?

b) on pose $Z = \frac{1}{1+X}$

$$\begin{aligned} E[Z] &= \sum_{h \geq 0} \frac{1}{1+h} \cdot e^{-\lambda} \frac{\lambda^h}{h!} = e^{-\lambda} \sum_{h \geq 0} \frac{\lambda^h}{h!(1+h)} \\ &= e^{-\lambda} \sum_{h \geq 0} \frac{\lambda^h}{(h+1)!} \\ &= \frac{e^{-\lambda}}{\lambda} \sum_{h \geq 0} \frac{\lambda^{h+1}}{(h+1)!} \\ &= \frac{e^{-\lambda}}{\lambda} \sum_{h \geq 0} \frac{\lambda^h}{h!} \\ &= \frac{e^{-\lambda}}{\lambda} \left(\sum_{h \geq 0} \frac{\lambda^h}{h!} - 1 \right) \\ &= \frac{1}{\lambda} - \frac{e^{-\lambda}}{\lambda} = \frac{1-e^{-\lambda}}{\lambda} \end{aligned}$$

($\forall \lambda \in \mathbb{R}$)

B) ALGÈBRE

Soit E un espace euclidien et soit $f \in \mathcal{L}(E)$ tel que : $\forall x, y \in E \quad x \perp y \Rightarrow f(x) \perp f(y)$.

But: On veut montrer qu'il existe $\lambda \in \mathbb{R}_+$: $\forall x \in E \quad \|f(x)\| = \lambda \|x\|$

* Utilisons l'induction. Soit $B = (e_1, \dots, e_n)$ une base orthonormée (une telle base existe car E est un espace euclidien, et donc de dimension finie).

Prenons $\forall i \in \{1, \dots, n\} \quad \lambda_i = \|f(e_i)\|^2$

On a :

linéarité de f

$$\langle f(e_i + e_j), f(e_i - e_j) \rangle = \langle f(e_i) + f(e_j), f(e_i) - f(e_j) \rangle$$

bilinéarité de $\langle \cdot, \cdot \rangle$
de symétrie

$$= \langle f(e_i), f(e_i) \rangle - \langle f(e_i), f(e_j) \rangle + \langle f(e_j), f(e_i) \rangle - \langle f(e_j), f(e_j) \rangle$$

$$= \|f(e_i)\|^2 - \|f(e_j)\|^2$$

$$= \lambda_i - \lambda_j \quad (*)$$

En particulier, $e_i + e_j \perp e_i - e_j \quad \forall i, j \in \{1, \dots, n\}$ ($\langle e_i + e_j, e_i - e_j \rangle = 0$)
 $\Rightarrow f(e_i + e_j) \perp f(e_i - e_j)$ par l'hypothèse (l'hypothèse)

$$\text{D'après } (*) = 0 \Rightarrow \lambda_i = \lambda_j \quad \forall i, j \in \{1, \dots, n\}$$

Notons donc $\lambda := \lambda_1 = \dots = \lambda_n$.

* Soit $x \in E$ quelconque, mettons $x = \sum_{i=1}^n e_i x_i$, $x_i \in \mathbb{R}$
On a $f(x) = f(\sum_{i=1}^n e_i x_i) = \sum_{i=1}^n f(e_i) x_i = \sum_{i=1}^n \lambda_i x_i$

On sait de plus que $\|f(x)\|^2 = \langle f(x), f(x) \rangle$.

$$\langle f(x), f(x) \rangle = \langle \sum_{i=1}^n f(e_i) x_i, \sum_{j=1}^n f(e_j) x_j \rangle$$

$$= \sum_{i,j=1}^n \langle x_i, x_j \rangle \langle f(e_i), f(e_j) \rangle$$

$$= \sum_{i,j=1}^n x_i x_j \langle f(e_i), f(e_j) \rangle = \sum_{i,j=1}^n x_i^2 \lambda_i = \sum_{i=1}^n x_i^2 \lambda$$

$$\lambda^2 \sum_{i=1}^n x_i^2 = \lambda^2 \|x\|^2$$

Donc $\|f(x)\| = \sqrt{f(x), f(x)}$

$$= \sqrt{\lambda^2 \|x\|^2}$$

$$= \lambda \|x\| \quad \square$$

C) ALGORITHMIQUE

exercice 1:

1) Algorithme (a):

L:

1	2	3	4	5	6	7
---	---	---	---	---	---	---

fact1

→

1	2	3	4	5	6	7
---	---	---	---	---	---	---

→

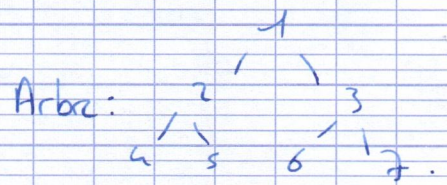
1	2	3	4	5	6	7
---	---	---	---	---	---	---

→

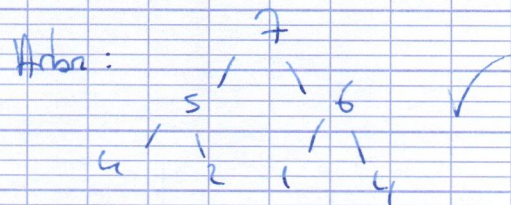
1	2	3	4	5	6	7
---	---	---	---	---	---	---

→

1	2	3	4	5	6	7
---	---	---	---	---	---	---



$$7 // 2 = 3$$



Algorithme (ii)

Idee: on insère comme fils (feuille) le dernier nombre à ajouter, puis on le fait monter à la bonne place. C'est plus simple directement avec la représentation d'un arbre binaire.

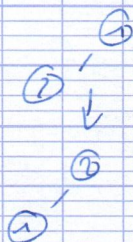
$T = [7, 1, 2, 3, 4, 5, 6, 7]$ (j'ajoute dans la suite $T(6)$).

• Insertion de 1: $[1]$

①

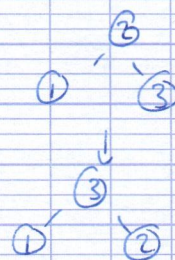
• Insertion de 2: $[1, 2]$

$\downarrow$
 $[2, 1]$



• Insertion de 3: $[2, 1, 3]$

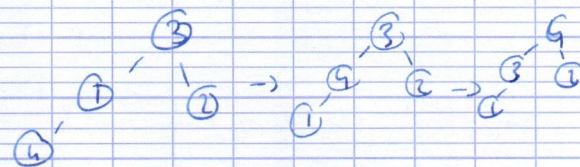
$\downarrow$
 $[3, 1, 2]$



• Insertion de 4: $[3, 1, 2, 4]$

$\downarrow$
 $[4, 1, 2, 3]$

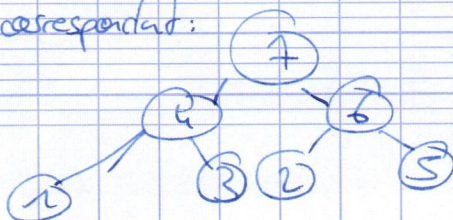
$\downarrow$
 $[4, 3, 2, 1]$



• Et la suite des états du tableau pour l'insertion de 5, 6 et 7 est:

$[4, 3, 2, 1, 5] \rightarrow [4, 5, 2, 1, 3] \rightarrow [5, 4, 2, 1, 3]$
 $\rightarrow [5, 4, 2, 1, 3, 6] \rightarrow [5, 4, 6, 1, 3, 2] \rightarrow [6, 4, 5, 1, 3, 2]$
 $\rightarrow [6, 4, 5, 1, 3, 2, 7] \rightarrow [6, 4, 7, 1, 3, 2, 5] \rightarrow [7, 4, 6, 1, 3, 2, 5]$

Arbre correspondant:



2) On vient justement de faire le pire cas: un tableau strictement croissant.

• On voit bien que dans ce cas, l'algorithme (i) est bien plus efficace.

* Complexité dans le pire cas de (ii):

→ Insérer le plus bas (feuille la + à droite)

→ Remonter à la racine.

Chaque opération est en $O(n)$, soit ici $O(\log n)$.

$$\sum_{h=1}^n \log h = \frac{1}{2} \log(n!) \in O(n \log n)$$

* Complexité de (i): $O(n)$ (cf. cours).

exercice 2: Considérons T un tableau d'entiers positifs.

1) def entasser_max(H, i):

$imax, left, right = i, gauche(i), droite(i)$

if ($left \leq \text{taille}(H)$ and $T[left] > T[imax]$):

$imax = left$

if ($right \leq \text{taille}(H)$ and $T[right] > T[imax]$):

$imax = right$

if ($imax \neq i$):

$T[imax], T[i] = T[i], T[imax]$

entasser_max($H, imax$)

return H .

2) Utilisons l'algorithme (i) de l'exercice 1.

```
def creer_tas_max(T):  
    for i in range(len(T)//2, 0, -1):  
        echanger_max(T, i)  
    return T
```

3) → récupérer puis retirer du tableau.

```
def extraire_max(T):  
    T = creer_tas_max(T)  $\leftarrow O(n)$   
    max = T[0]  
    T[0] = max T[-1]  
    echanger_max(T, 1)  $\leftarrow O(\log n)$   
    T[0] = T[0] - 1  
    return T
```

$O(n)$

NB: si T était supposé déjà un tas, on aurait $O(\log n)$.